Course Management System

Requirements Analysis Document

Version 4

CSCI 4711 Software Engineering
Fall 2025
Augusta University

Team Members

Caleb Acree
Thomas Beeson
Jordan Pruner
Derek Richins

Table of Contents

1 INTRODUCTION 3

1.1 SCOPE OF SYSTEM.....3

2 REQUIREMENTS OF SYSTEM.....4

2.1 FUNCTIONAL REQUIREMENTS 4
2.2 NON-FUNCTIONAL REQUIREMENTS 5
2.3 USE CASES.....6
2.4 USE CASE DESCRIPTIONS.....7
2.5 REQUIREMENTS ANALYSIS 13

3 USER INTERFACE MOCKUPS 20

3.1 LOGIN 20
3.2 LOGOUT 22
3.3 REGISTER FOR CLASS.....23
3.4 ENTER GRADE 24

4 OBJECT DESIGN 24

4.1 OBJECT INTERACTION 25
4.2 CLASS DESIGN.....29
4.3 DATABASE DESIGN.....30

5 SYSTEM DESIGN ..... **Error! Bookmark not defined.**

6 APPENDIX 32

6.1 APPENDIX A – SOURCE CODE 32

1 INTRODUCTION

1.1 SCOPE OF SYSTEM

The Course Management System is a digital platform designed to support the academic operation of an education institute. It provides functionality for two primary actors: Students, who register for classes, and Teachers, who manage the grades for their enrolled students.

The system supports both actors by maintaining accurate, consistent records of academic data. It acts with a centralized database that stores student enrollment, course information, and grade records. Access to this database is provided through a unified central portal for all actors.

The Course Management System includes functionality for secure authentication of users, course registration and management, grade entry, and retrieval of academic history. Students are able to register for classes and view course details. Teachers can manage the grades for their courses and view academic information of their students. All stored data is internal to the system and self-contained.

2 REQUIREMENTS OF SYSTEM

2.1 FUNCTIONAL REQUIREMENTS

- Startup: This includes the initial set of tasks that the system needs to perform once it is started. The tasks performed include:
    - Performing application configurations and initializing the database with data
    - Loading an initial set of data (e.g., courses)
- Login: Both user types should be required to login to access some set of the system's functionality.
    - Every user login must be saved in the database.
- Logout: Any user that is logged in should be able to logout.
    - Every user logout must be saved in the database.
- EnterGrade: Teachers, once logged in, should be able to select a course and input grades for each student registered for that course.
- RegisterForClass: Students, once logged in, should be able to select classes to see information, such as meeting times and locations, and register for classes.

2.2NON-FUNCTIONAL REQUIREMENTS

- Platform
    - Target operating system is Microsoft Windows
    - System should be implemented using the Java programming language
    - System should not be web-based, nor should it utilize a web browser
- Usability
    - System should implement a graphical user interface (GUI)
- Security
    - All user input should be validated/sanitized appropriately to prevent SQL injection attacks
    - User data should never be stored in plaintext in the database. User data should be encrypted.

2.3USE CASES

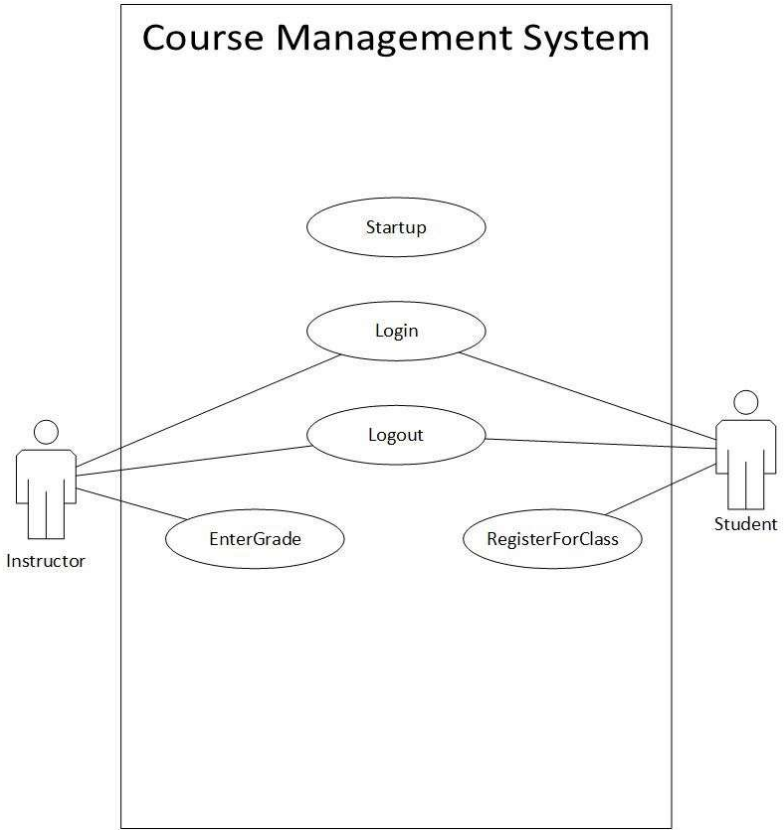

Figure 2.1 Use Case Diagram

2.4 USE CASE DESCRIPTIONS

Use case name	Startup
Participating actors	
Pre-condition	
Flow of events	1. This use case is initiated when the application is launched. **2. The system reads the configuration file, initializes the database, and displays the Login.**
Post condition	The Login is displayed.
Security requirements	Username and password are not stored as plaintext in the database.

Table 2.2: Startup

Use case name	Login
Participating actors	Initiated by Student or Instructor
Pre-condition	Startup completed and database loaded
Flow of events:	1. The user fills in their username and password, and hits submit on the LoginForm. **2. The course management system receives the form and verifies the username and password. If the credentials are correct, InstructorMenu is displayed for Instructors or StudentMenu is displayed for Students.**
Post condition	The appropriate GUI is displayed for the user.
Security requirements	Credentials should not be displayed as numbers or letters in the textbox during entry. User inputs are verified against their specification. User inputs are validated to prevent SQL injection.

Table 2.3: Successful Login

Use case name	Login
Participating actors	Initiated by Student or Instructor
Pre-condition	Startup completed and database loaded
Flow of events:	1. The user fills in their username and password, and hits submit on the LoginForm. **2. The course management system receives the form and verifies the username and password. If the credentials are incorrect, an error message is displayed on the LoginForm.**
Post condition	
Security requirements	Credentials should not be displayed as numbers or letters in the textbox during entry. User inputs are verified against their specification. User inputs are validated to prevent SQL injection.

Table 2.4: Failed Login

Use case name	RegisterForClass
Participating actors	Student
Pre-condition	User must have logged in using student credentials
Flow of events	1. Student selects a course they wish to register for from their StudentMenu. **2. The system displayed retrieves the available sections and locations and displays them on the RegistrationForm.** 3. System saves the course registration and displays the StudentMenu with updated course registration information.
Post-condition	An updated list of courses the student has registered for is displayed.
Security requirements	When the submit button is clicked, student/course information is encrypted and stored in the database.

Table 2.5: RegisterForClass

Use case name	EnterGrade
Participating actors	Initiated by Instructor
Pre-condition	Instructor Login Completed
Flow of events	1. From the InstructorMenu, Instructor selects a course for which grading is not yet completed. **2. The system retrieves the enrollment for the course including all graded course work and assigned grades. This information is displayed on the GradeForm.** 3. Instructor enters the grades for each student and submits the form. **4. The system saves the grades and displays the InstructorMenu with updated grading status for each course.**
Post condition	
Security requirements	Instructor input will be sanitized to protect against any malicious input. Session management ensures only authenticated instructors can enter or update grades for their course.

Table 2.6: EnterGrade

Use case name	Logout
Participating actors	Initiated by Student or Instructor
Pre-condition	Actors are logged in.
Flow of events	1. User clicks the logout button on their MainMenu. **2. The system saves the logout event to the database.**
Post condition	The User's session is terminated, and the login page is displayed.
Security requirements	

Table 2.7: Logout

2.5REQUIREMENTS ANALYSIS

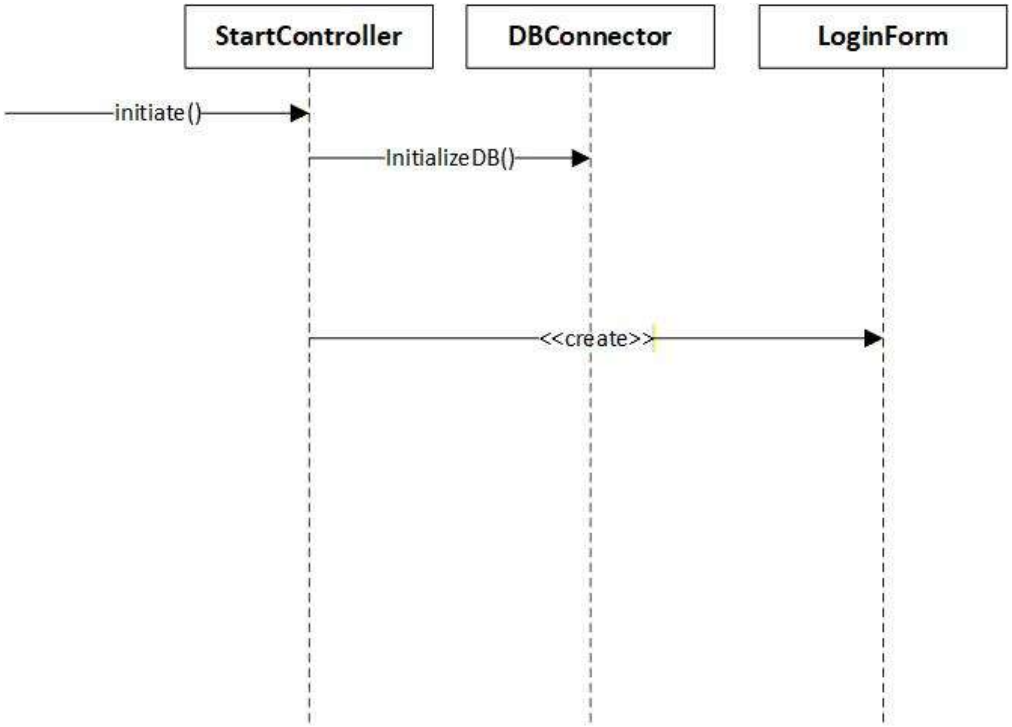

Figure 2.8: Start up

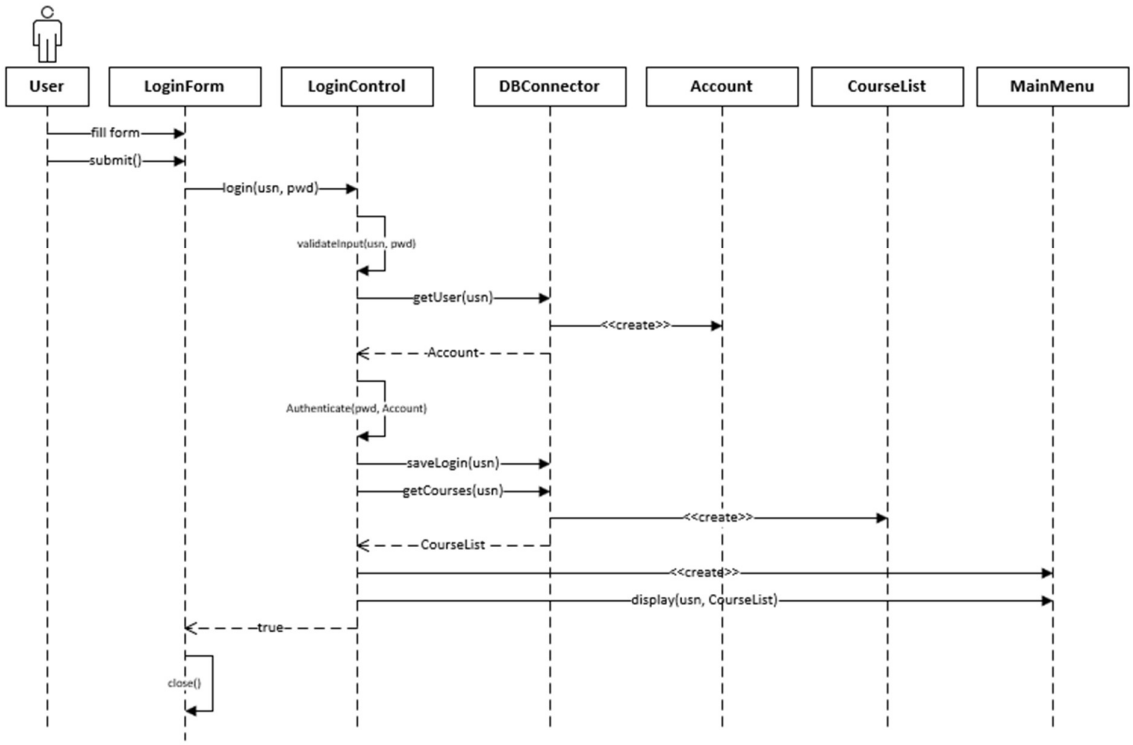

Figure 2.9-1: Successful Login

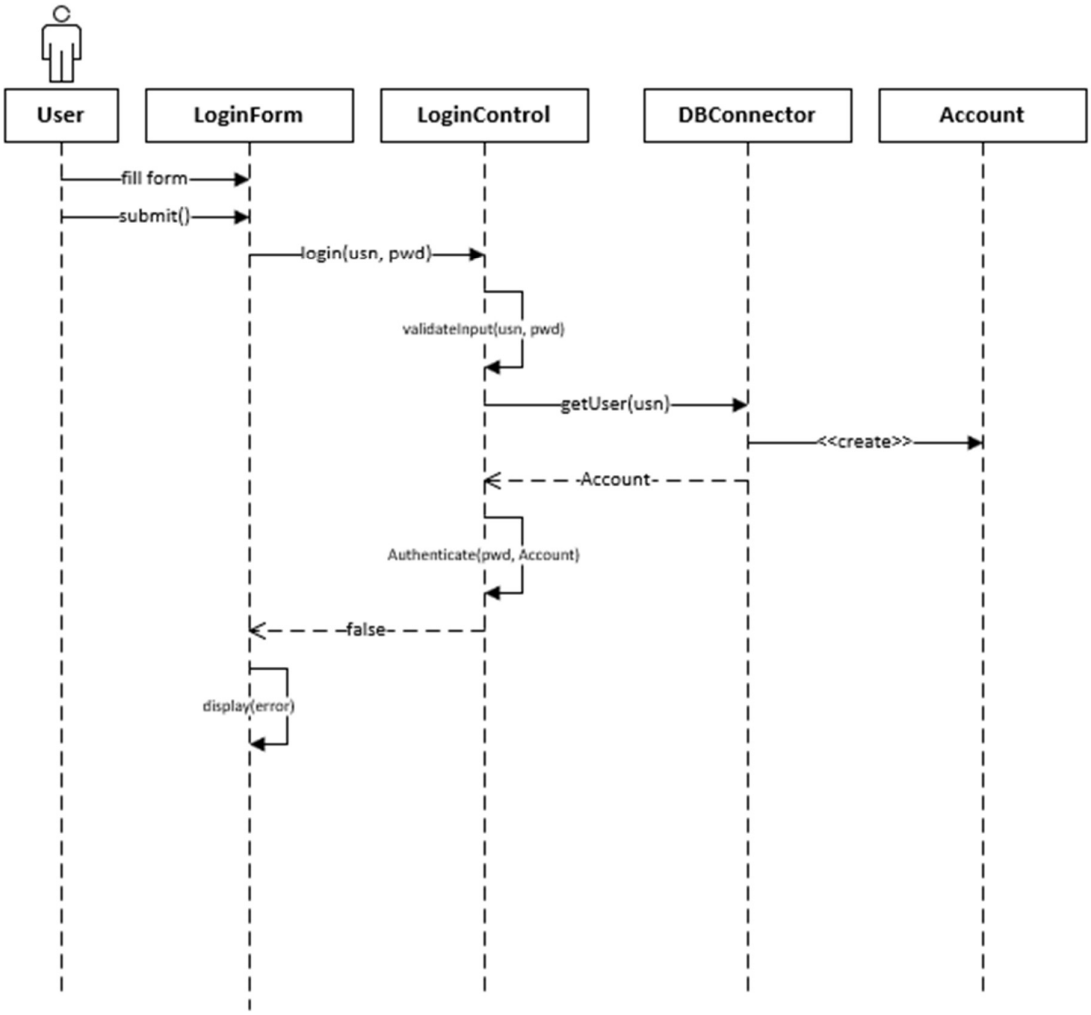

Figure 2.9-2: Unsuccessful Login

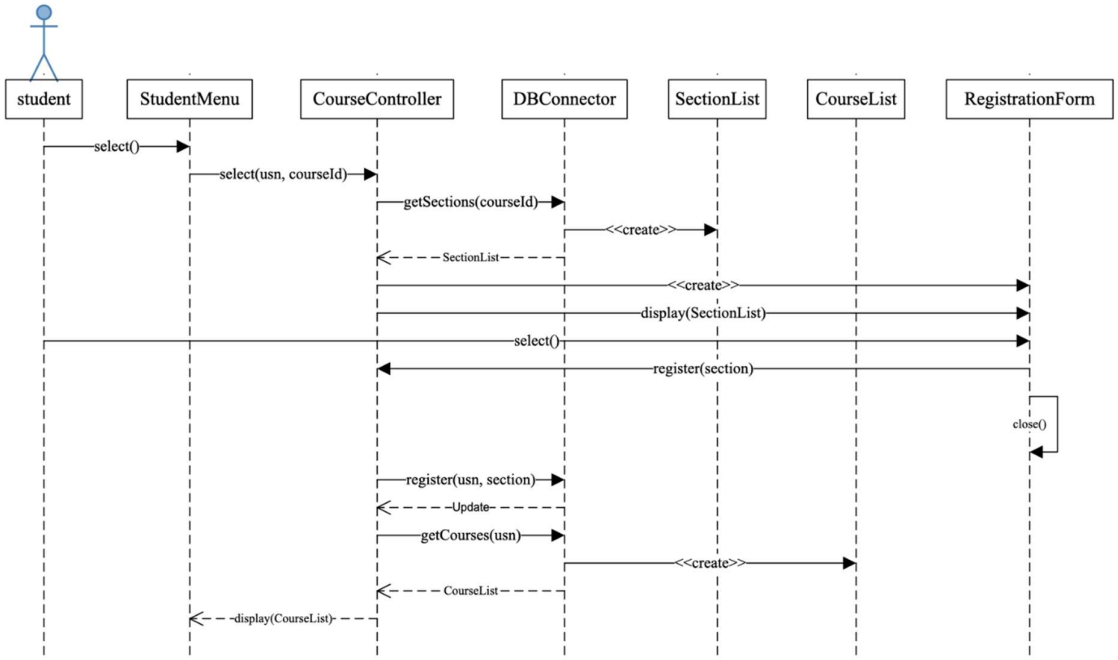

Figure 2.10: Register for class

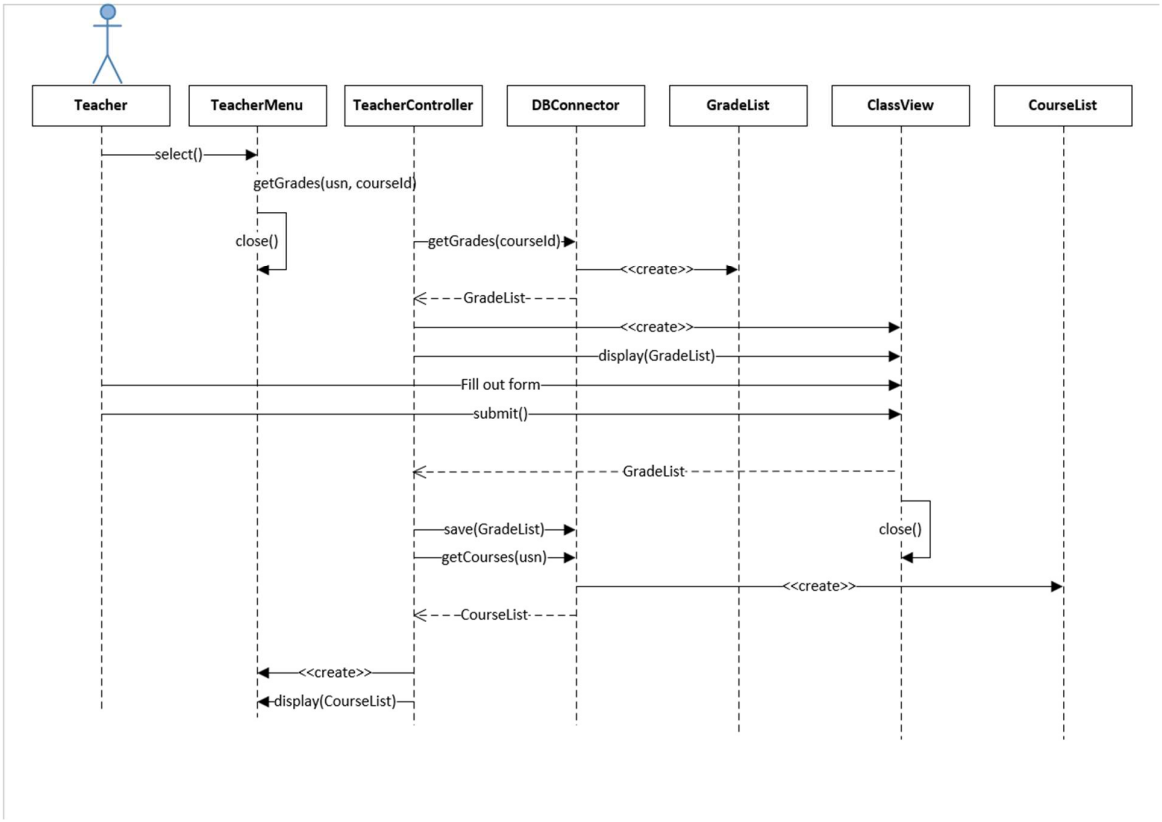

Figure 2.11: Enter grade

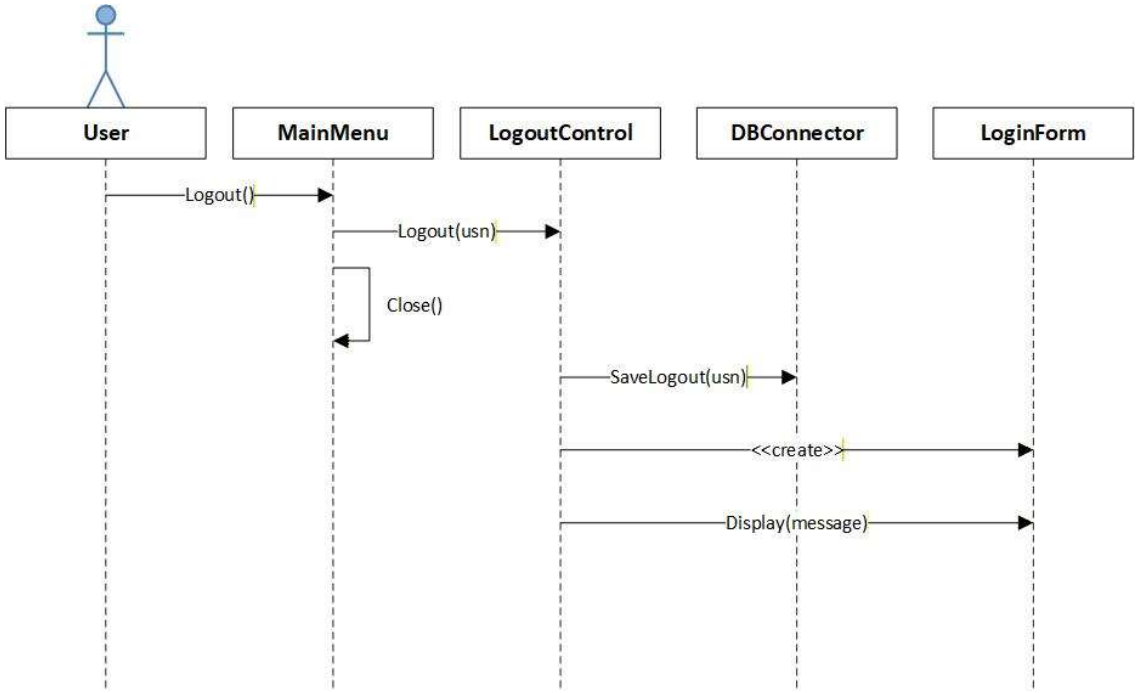

Figure 2.12: Log out

3 USER INTERFACE MOCKUPS

3.1 LOGIN

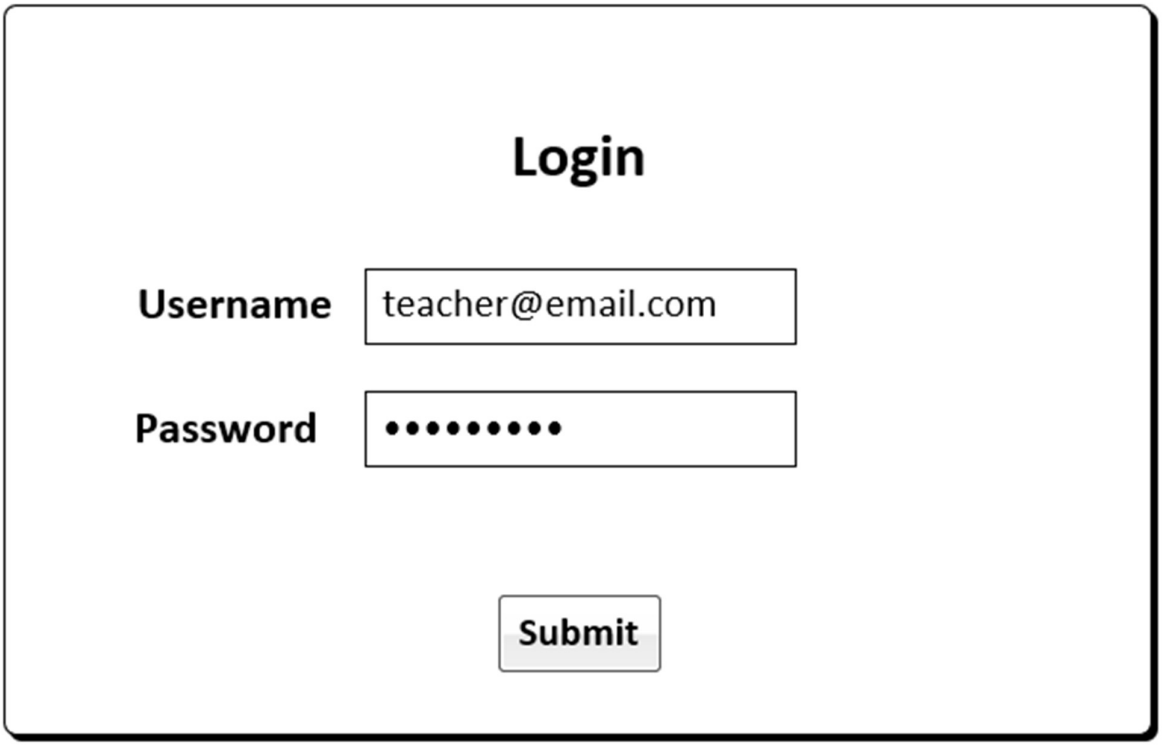

Figure 3.1: Login

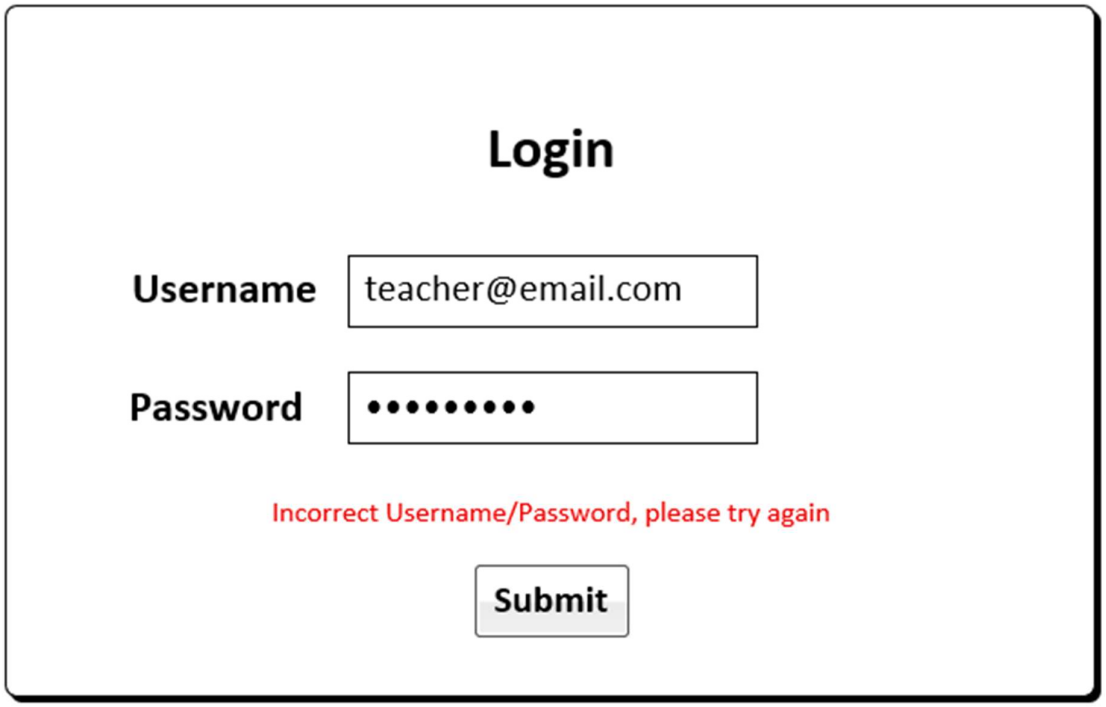

Figure 3.2: Login failure

3.2LOGOUT

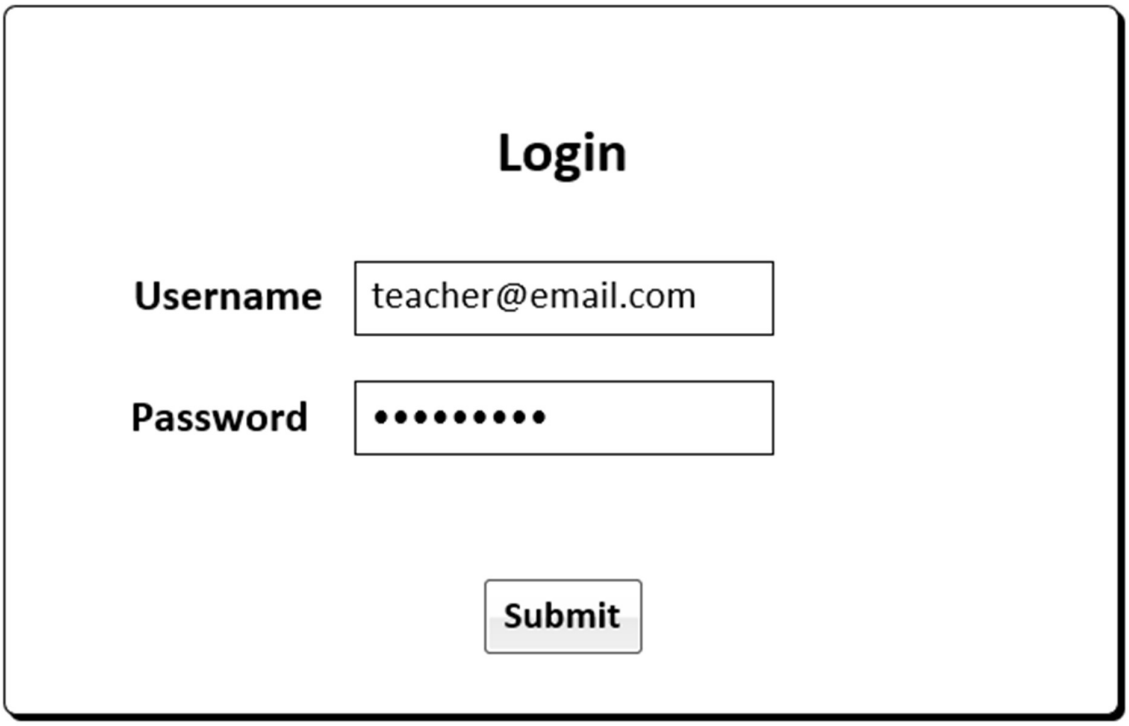

Figure 3.3: Logout

3.3REGISTER FOR CLASS

Student Name

Major

Logout

Available Courses

Course Number	Course Name	
CSCI 3170	Computer Ogranization	Select
CSCI 3370	Low-Level Program Languages	Select
CSCI 3400	Data Structures	Select
CSCI 3410	Database Systems	Select
CSCI 3500	Theory of Computation	Select

Registered Courses

Course Number	Course Name	Time	Location
MATH 3280	Intro to Stats & Data Analysis	T/Th 11-12:15	EC 1215

Post-login landing page

Figure 3.4: Student landing page

Course Title

Course Description

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Section	Time	Location	
CSCI 3170 A	T/Th 11:00-12:15	GCC 2600	Register
CSCI 3170 B	T/Th 2:30-3:45	GCC 2300	Register
CSCI 3170 C	M/W 2:00-3:15	GCC 2301	Register

Pop-up window displayed after selecting a class

Figure 3.5: Pop-up window displayed after selecting a class

3.4ENTER GRADE

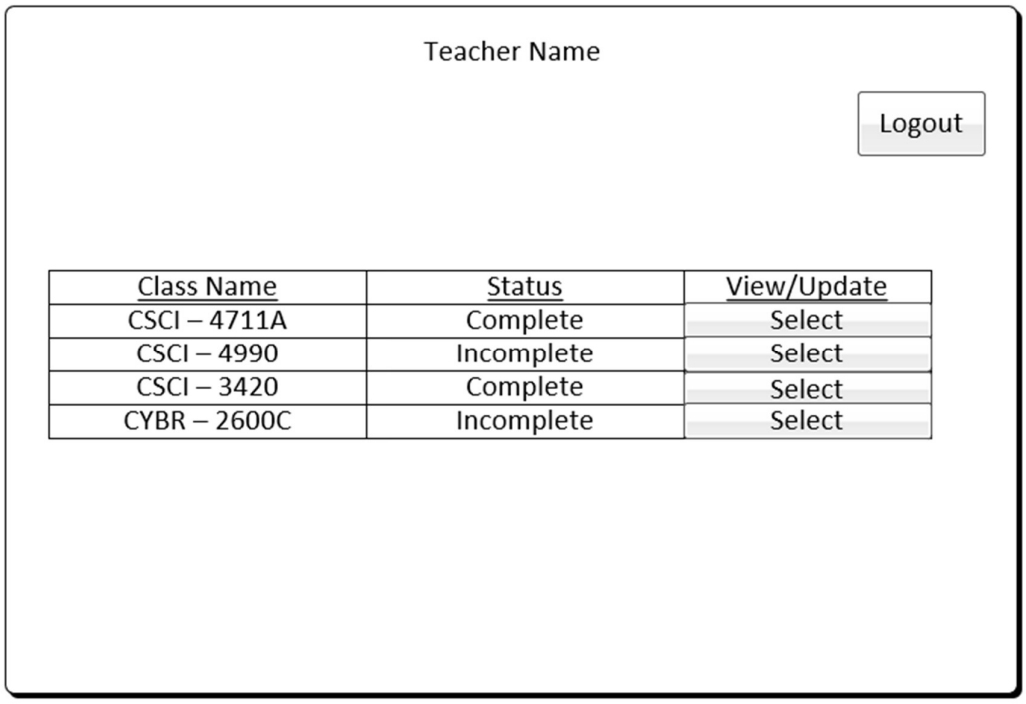

Figure 3.6: Teacher UI

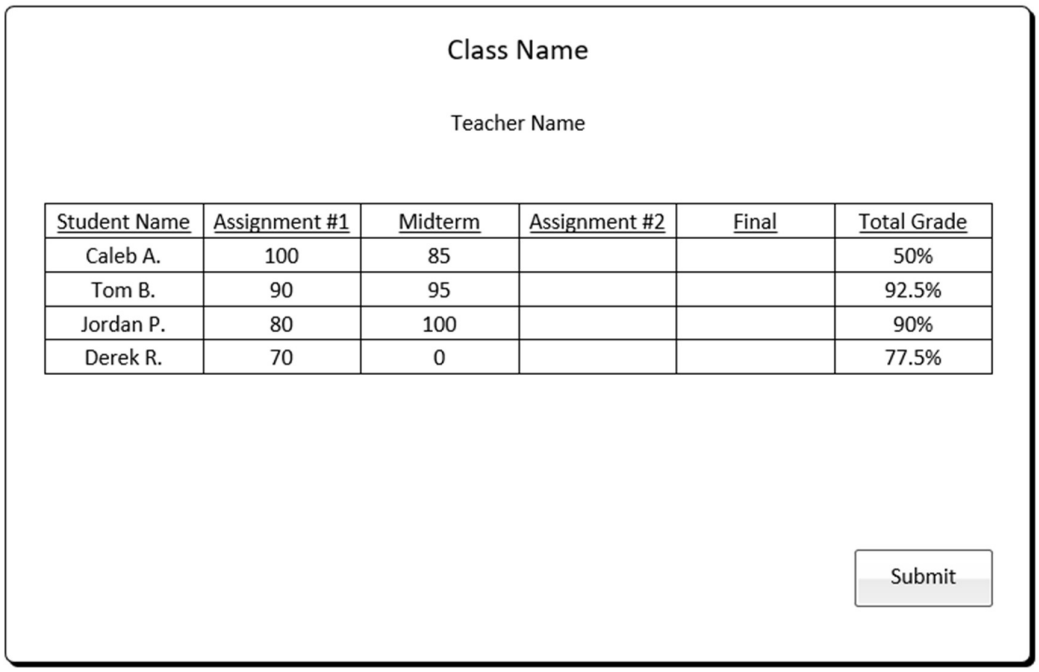

Figure 3.7: Grading UI

4 OBJECT DESIGN

4.1 OBJECT INTERACTION

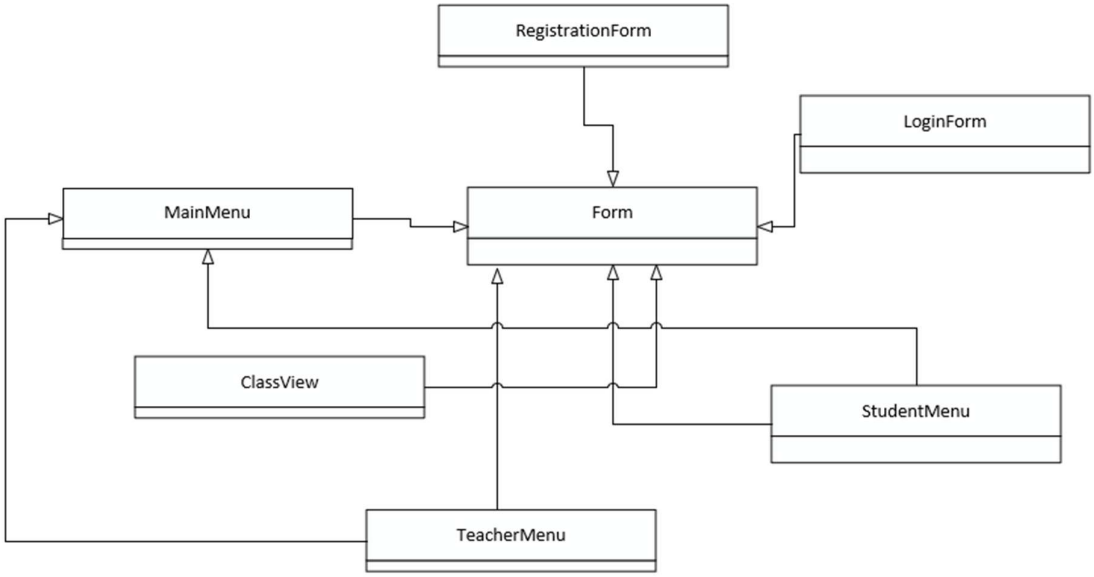

Figure 4.1: Class Diagram: Boundary

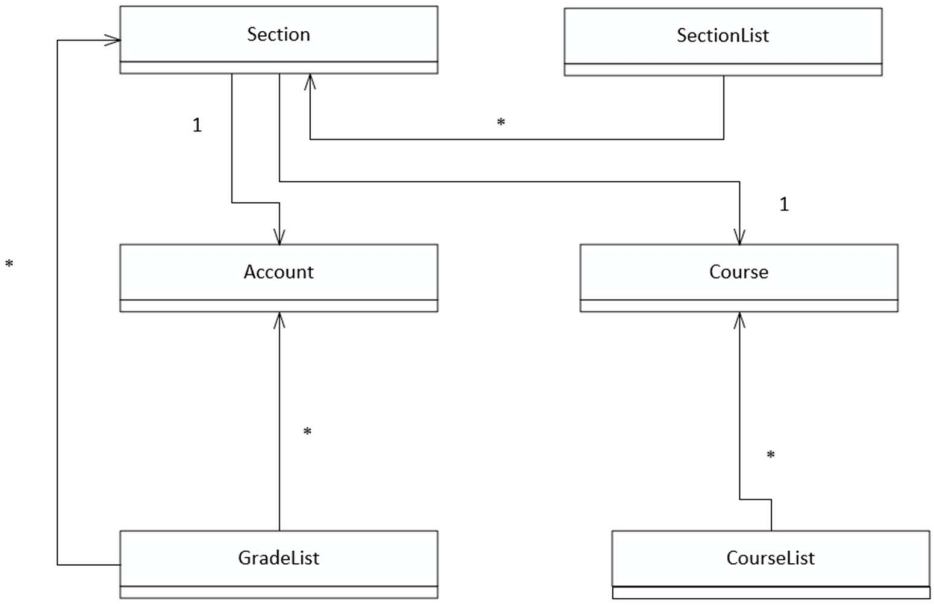

Figure 4.2 Class Diagram: Entity

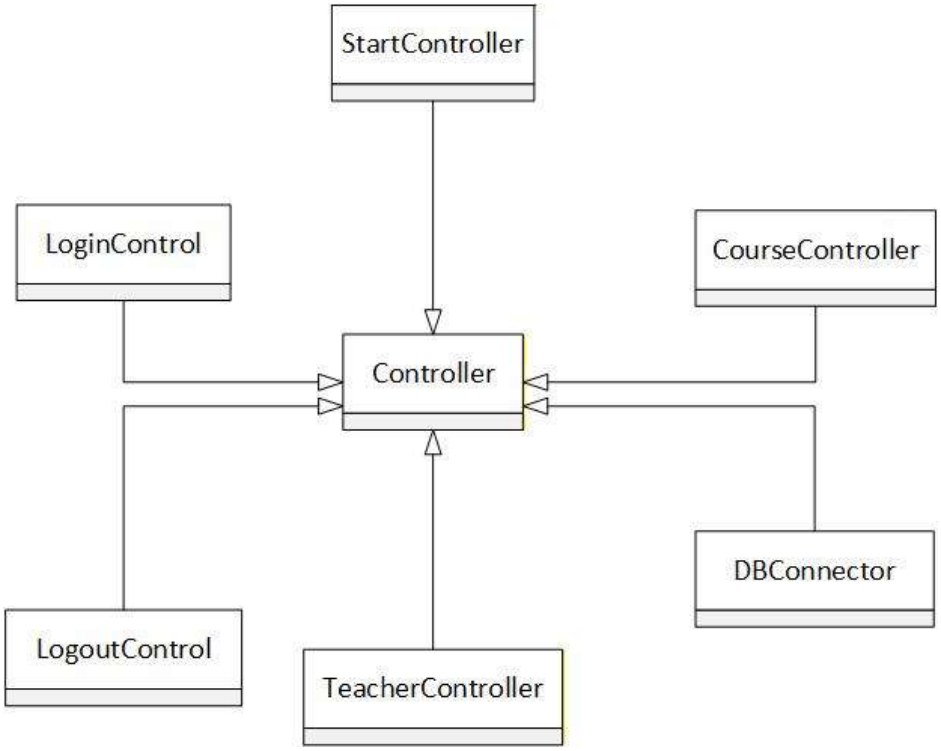

Figure 4.3 Class Diagram: Control

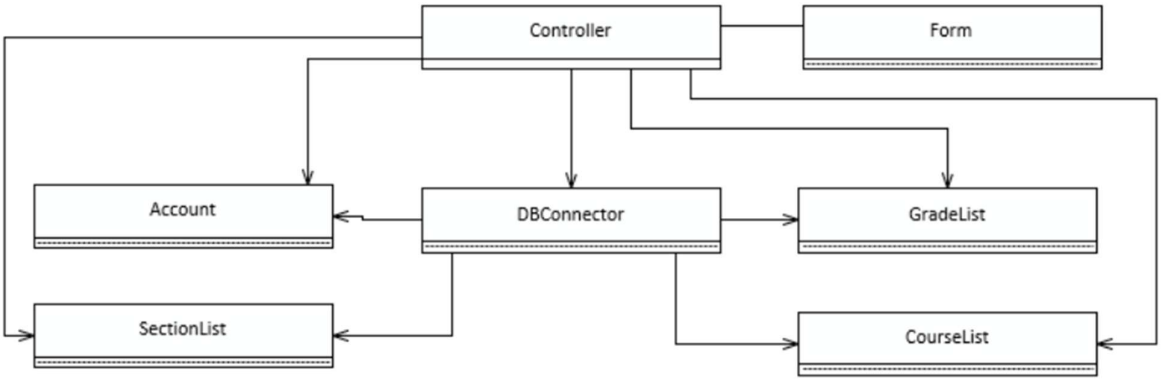

Figure 4.4 Class Diagram

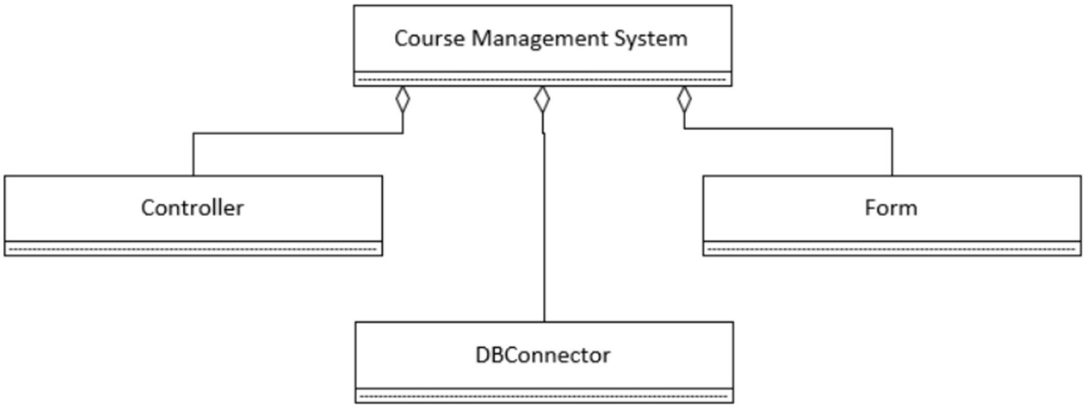

Figure 4.5 Class Diagram

4.2 CLASS DESIGN

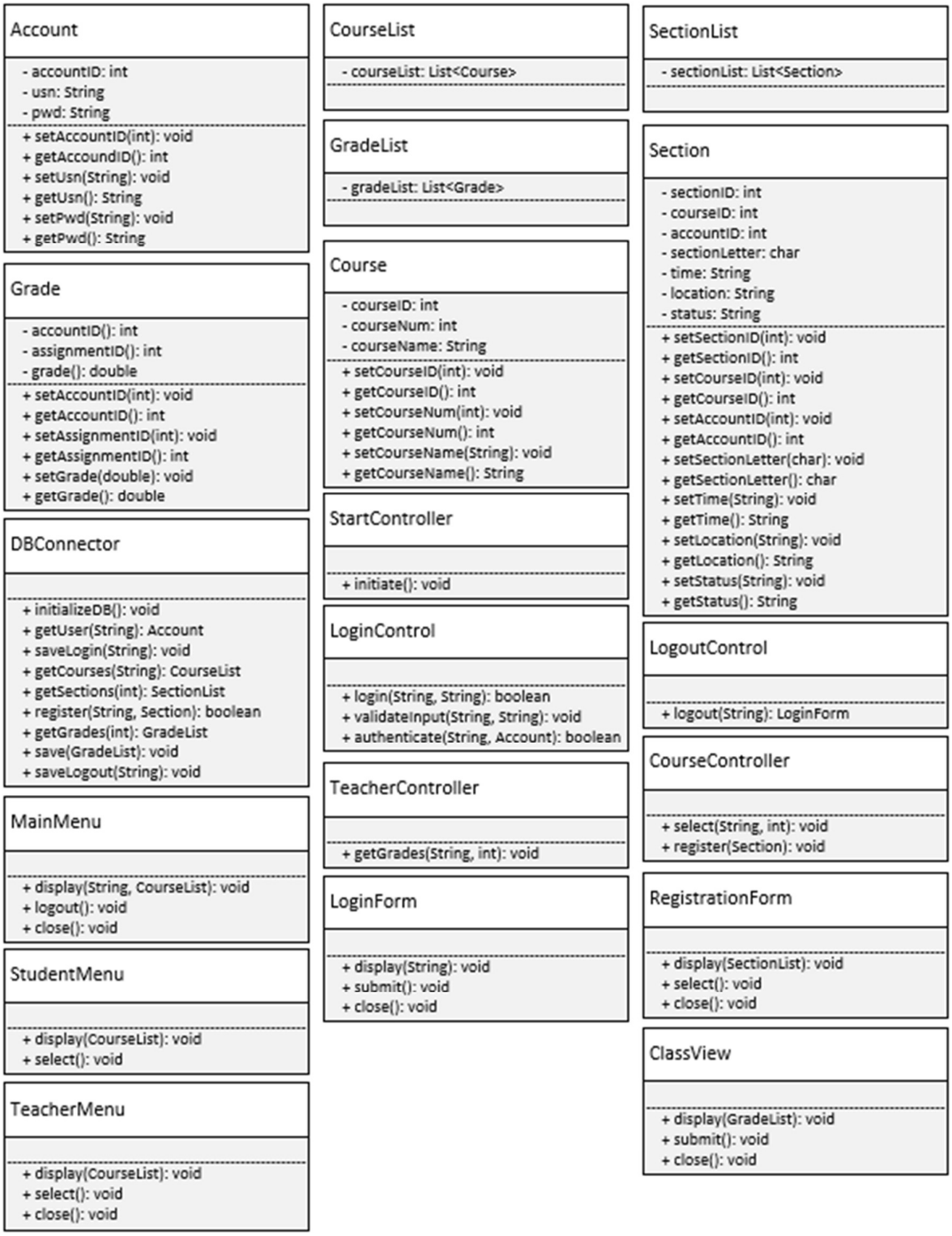

Figure 4.6 Class UML Diagrams

4.3 DATABASE DESIGN

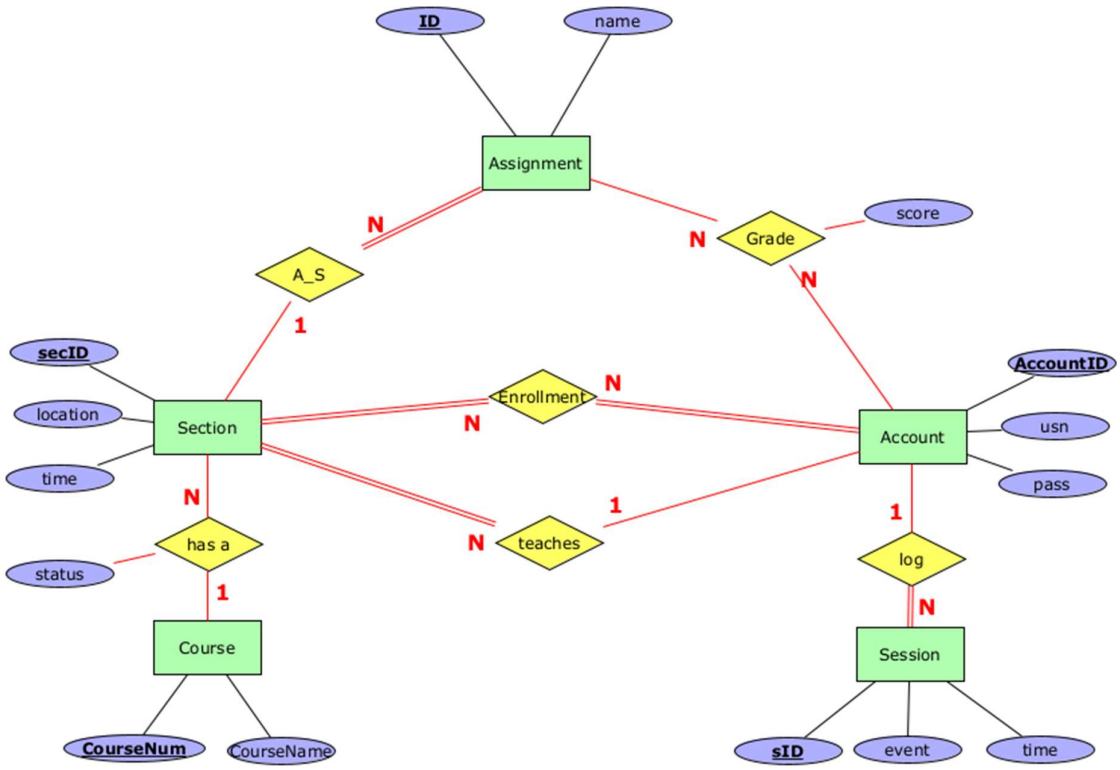

Figure 4.7 Entity Relationship Diagram

RELATIONAL MODEL

Account (accountID, usn, pwd)

Course (courseID, courseNum, courseName)

Section (secID, courseID^{FK}, location, time, status, accountID^{FK})

Assignment (asgmtID, name, secID^{FK})

Grade (gradeID, asgmntID^{FK}, accountID^{FK}, score)

Session (sID, event, time, accountID^{FK})

Enrollment (enrollID, secID^{FK}, accountID^{FK})

5 SYSTEM DESIGN

5.1 SUBSYSTEM DECOMPOSITION

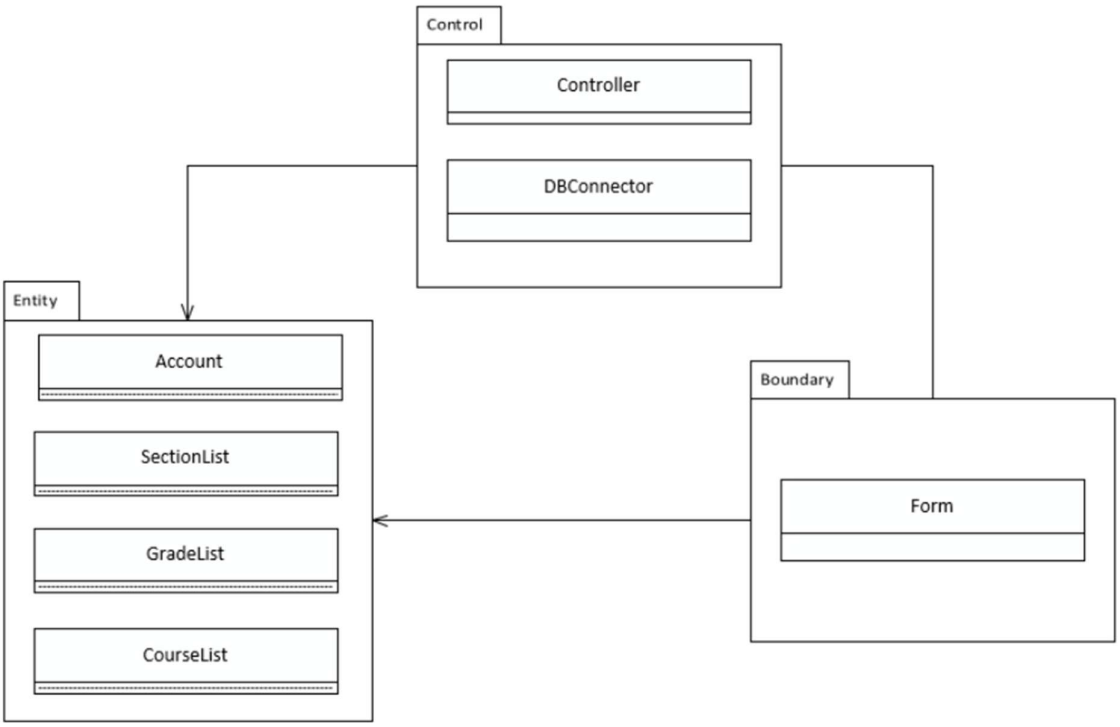

Figure 5.1: Subsystems

6 APPENDIX

6.1 APPENDIX A – SOURCE CODE

```
package Entity;

public class Account {

    private int accountID;
    private String usn;
    private String pwd;

    public Account() {
    }

    public void setAccountID(int accountID) {
        this.accountID = accountID;
    }

    public int getAccountID() {
        return accountID;
    }

    public void setUsn(String usn) {
        this.usn = usn;
    }

    public String getUsn() {
        return usn;
    }

    public void setPwdHash(String pwd) {
        this.pwd = pwd;
    }

    public String getPwdHash() {
        return pwd;
    }
}
```

```java
package Entity;

public class Course {

    private int courseID;
    private int courseNum;
    private String courseName;

    public Course() {
    }

    public void setCourseID(int courseID) {
        this.courseID = courseID;
    }

    public int getCourseID() {
        return courseID;
    }

    public void setCourseNum(int courseNum)  {
        this.courseNum = courseNum;
    }

    public int getCourseNum() {
        return courseNum;
    }

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }

    public String getCourseName() {
        return courseName;
    }
}
```

```java
package Entity;

import java.util.ArrayList;

public class CourseList {

    private ArrayList<Course> courses;

    public CourseList() {
        courses = new ArrayList<>();
    }

    public void add(Course c) {
        courses.add(c);
    }

    public Course getCoursesByID(int id) {
        for (Course c : courses) {
            if (c.getCourseID() == id) {
                return c;
            }
        }
        return null;
    }

    public ArrayList<Course> getAll() {
        return courses;
    }

    public int size() {
        return courses.size();
    }
}
```

```
package Entity;

public class Section {

    private int sectionID;
    private int courseID;
    private int accountID;
    private String time;
    private String location;
    private String status;

    public Section() {
    }

    public void setSectionID(int sectionID) {
        this.sectionID = sectionID;
    }

    public int getSectionID() {
        return sectionID;
    }

    public void setCourseID(int courseID) {
        this.courseID = courseID;
    }

    public int getCourseID() {
        return courseID;
    }

    public void setAccountID(int accountID) {
        this.accountID = accountID;
    }

    public int getAccountID() {
        return accountID;
    }

    public void setTime(String time) {
        this.time = time;
    }

    public String getTime() {
        return time;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public String getLocation() {
        return location;
    }

    public void setStatus(String status) {
```

```
        this.status = status;
    }

    public String getStatus() {
        return status;
    }
}
package Entity;

import java.util.ArrayList;

public class SectionList {

    private ArrayList<Section> sections;

    public SectionList() {
        sections = new ArrayList<>();
    }

    public void add(Section s) {
        sections.add(s);
    }

    public Section getSectionsByID(int id) {
        for (Section s : sections) {
            if (s.getSectionID() == id) {
                return s;
            }
        }
        return null;
    }

    public ArrayList<Section> getAll() {
        return sections;
    }

    public int size() {
        return sections.size();
    }
}
```

```java
package Entity;

public class Grade {

    private int gradeID;
    private int asgmtID;
    private int accountID;
    private double score;

    public Grade() {
    }

    public void setGradeID(int gradeID) {
        this.gradeID = gradeID;
    }

    public int getGradeID() {
        return gradeID;
    }

    public void setAsgmtID(int asgmtID) {
        this.asgmtID = asgmtID;
    }

    public int getAsgmtID() {
        return asgmtID;
    }

    public void setAccountID(int accountID) {
        this.accountID = accountID;
    }

    public int getAccountID() {
        return accountID;
    }

    public void setScore(double score) {
        this.score = score;
    }

    public double getScore() {
        return score;
    }
}
```

```java
package Entity;

import java.util.ArrayList;

public class GradeList {

    private ArrayList<Grade> grades;

    public GradeList() {
        grades = new ArrayList<>();
    }

    public void add(Grade g) {
        grades.add(g);
    }

    public Grade getgradesByID(int id) {
        for (Grade g : grades) {
            if (g.getGradeID() == id) {
                return g;
            }
        }
        return null;
    }

    public ArrayList<Grade> getAll() {
        return grades;
    }

    public int size() {
        return grades.size();
    }
}
```